Classic planning with PDDL

Aleksandra Słomska
Zofia Narloch

March 6, 2026

1 Exercise 1.1: Emergency Service Logistics, Initial Release

```
( PICK-UP-CRATE CRATE1 DEPOT DRONE1 RIGHT )
( DRONE-MOVE DEPOT ROOM1 DRONE1 )
( DROP-CRATE ROOM1 PERSON1 CRATE1 RIGHT FOOD DRONE1 )
```

Listing 1: Outcome of the first problem - one person and one box

```
( PICK-UP-CRATE CRATE1 DEPOT DRONE1 RIGHT )
( PICK-UP-CRATE CRATE2 DEPOT DRONE1 LEFT )
( DRONE-MOVE DEPOT ROOM1 DRONE1 )
( DROP-CRATE ROOM1 PERSON1 CRATE1 RIGHT FOOD DRONE1 )
( DROP-CRATE ROOM1 PERSON2 CRATE2 LEFT MEDICINE DRONE1 )
```

Listing 2: Outcome of the second problem - two people and three boxes

2 Exercise 1.2: Problem Builder in Python

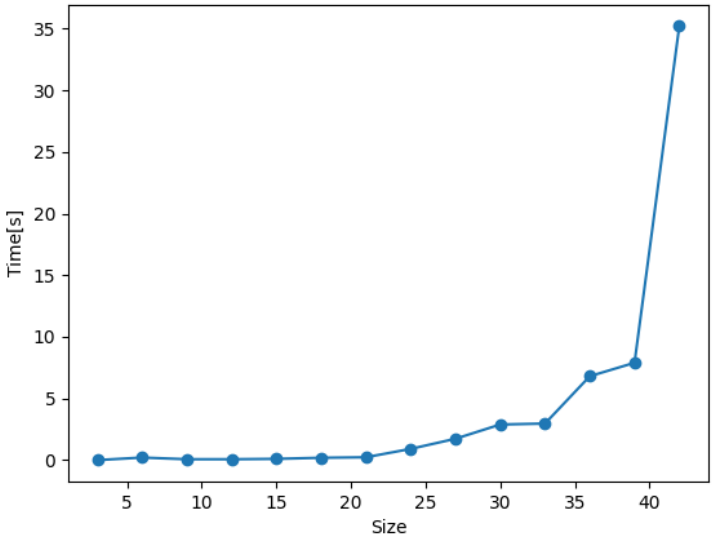

Figure 1: Size of the problem and the time required to find a solution in the tests

3 Exercise 1.3: Performance Comparison of Search Algorithms and Heuristics

3.1 First Part

Algorithm	Heuristic	Largest Solved Size	Time (s)	Plan Length	Optimal
BFS	None	7	24.59	22	Yes
IDS	None	3	0.71	10	Yes
A*	hMAX	5	9.95	17	Yes
GBFS	hMAX	7	10.94	26	No

Table 1: Performance Comparison of Search Algorithms and Heuristics (Exercise 1.3.1)

The results of the performance of every algorithm show that only GBFS with heuristic hMAX is not optimal. The reason for that is the longest plan length. This is because GBFS relies on the heuristic to rush toward the goal quickly rather than exhaustively searching for the perfect path. The same size of the

problem was solved by the BFS algorithm without any heuristics. However, it took more time, because it blindly checks every possible combination of moves. On the other hand, the A* algorithm only reached size 5 because the calculation of the hMAX heuristic for every node caused it to time out. Algorithm IDS solved the smallest problem of them all. IDS intentionally wipes its memory and starts over every time it increases its search depth, so it concentrates on memory efficiency.

In this part every action in domain file has the same cost, that is why algorithms like BFS guarantee the shortest path by expanding all nodes at the current search depth before moving deeper. Secondly, at every step, the drone has many valid moves, which creates many possibilities, which can cause problems for some algorithms like BFS.

3.2 Second Part

Algorithm	**Heuristic**	**Time (s)**	**Plan Length**
GBFS	NONE	0.37	26
GBFS	hMAX	10.68	26
GBFS	hFF	0.28	26
GBFS	hADD	0.35	27
GBFS	LANDMARK	0.29	28
EHC	NONE	0.31	28
EHC	hMAX	Timeout	$>$ 60s
EHC	hFF	0.39	28
EHC	hADD	0.44	28
EHC	LANDMARK	0.27	31

Table 2: Performance Comparison of GBFS and EHC with Various Heuristics (Exercise 1.3.2)

Based on the results from Exercise 1.3.1, the largest problem that Greedy Best-First Search (GBFS) could handle within the 1-minute time limit was size 7. This problem size was generated and tested for the algorithms GBFS and EHC with heuristics hMAX, hFF, hADD and Landmark. Only algorithm EHC with heuristic hMAX exceeded the time limit. The GBFS solved a size 7 problem with hMAX in under 11 seconds.

EHC is more memory-efficient than GBFS, since it discards alternative paths once it makes a choice. It causes the algorithm to be fast and use minimal memory. However, it causes problems when the chosen path reaches dead-end, since it can't go back. The GBFS keeps all values in memory, so it can return if the path is unpromising, but at cost of taking much more memory.

The FF planer intelligently combines the strengths of both algorithms to maximize speed and reliability. The FF planner primarily relies on EHC paired with the hFF heuristic. However, if EHC gets trapped in a dead-end, the planner

automatically changes to GBFS. The result of that is the planner can backtrack out of the dead-end, guaranteeing that a solution will eventually be found.

3.3 Third Part

Algorithm	**Heuristic**	**Time (s)**	**Plan Length**
A*	NONE	6.51	17
BFS	NONE	0.77	17
IDS	NONE	Timeout	> 60s
A*	hMAX	9.18	17
A*	LMCUT	47.76	17

Table 3: Performance of Optimal Planners and Admissible Heuristics (Exercise 1.3.3)

An admissible heuristic is one that never overestimates the true cost to reach the goal, guaranteeing that an algorithm like A* will find an optimal solution. Among the heuristics implemented in Pyperplan, hMAX and LMCUT (Landmark-Cut) are admissible.

The results show that almost every algorithm has a plan length equal to 17. The best-performing algorithm was BFS without any heuristics. The reason for that is the same cost for every action. Moreover, adding the heuristics to algorithm A* resulted in exceeding the time by about 1.4 times for hMAX and over 7 times for LMCUT. It is because of the necessity to calculate these complex heuristics at every node.

Finally, IDS timed out because a plan length of 17 steps requires it to wipe its memory and completely regenerate the search tree from scratch 17 times, which causes it to exceed the 1-minute limit.

4 Exercise 2.1: Emergency Service, Carriers

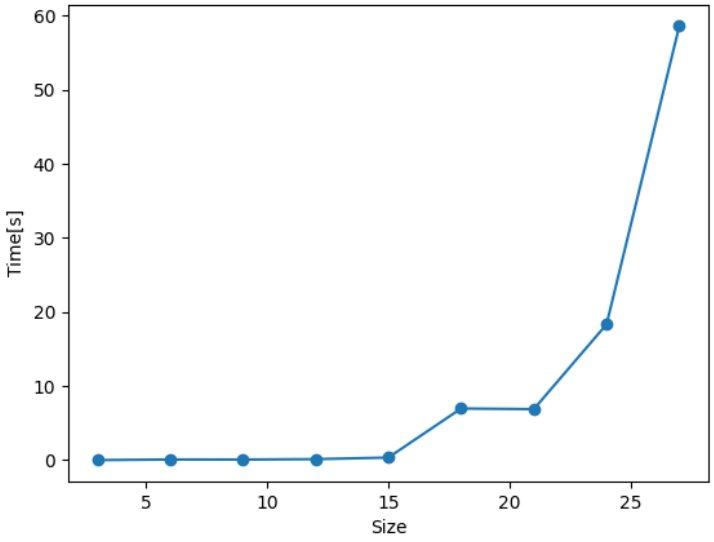

Figure 2: Size of the problem and the time required to find a solution in the tests

Algorithm	**Heuristic**	**Time P1 (s)**	**Time P2 (s)**	**Plan P1**	**Plan P2**
GBFS	NONE	0.37	0.62	26	23
GBFS	hMAX	10.68	Timeout	26	> 60s
GBFS	hFF	0.28	0.62	26	23
GBFS	hADD	0.35	0.54	27	26
GBFS	LANDMARK	0.29	0.42	28	28
EHC	NONE	0.31	0.54	28	23
EHC	hMAX	Timeout	Timeout	> 60s	> 60s
EHC	hFF	0.39	0.47	28	23
EHC	hADD	0.44	1.18	28	30
EHC	LANDMARK	0.27	0.43	31	31

Table 4: Performance Comparison of GBFS and EHC

Algorithm	Heuristic	Time P1 (s)	Time P2 (s)	Plan P1	Plan P2
A*	NONE	6.51	4.71	17	14
BFS	NONE	0.77	0.64	17	14
IDS	NONE	Timeout	Timeout	> 60s	> 60s
A*	hMAX	9.18	8.59	17	14
A*	LMCUT	47.76	42.49	17	14

Table 5: Performance of Optimal Planners

By adding the carriers the plan length was improved. The reason for that is the drone does not have to go back to depot to take more crates, he is able to take more at once. The time also improved in some cases especially in algorithm A with or without heuristics and BFS. In algorithms GBFS and EHC time has improved with most of the heuristics. This significant improvement in execution time is a result of modeling the carrier capacity using numerical tracking rather than slots.

5 Exercise 2.2: Emergency Service, Action Costs

Test run with lama-first alias

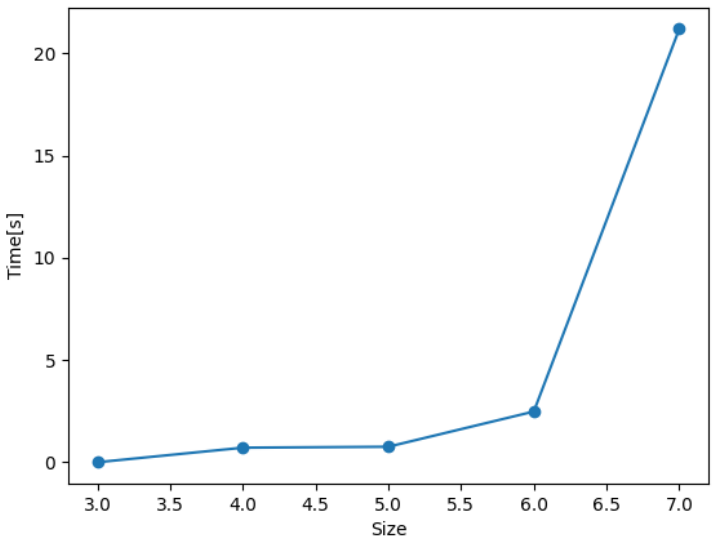

Figure 3: Size of the problem and the time required to find a solution in the tests